

# Module 5

## Advanced UVM Concepts

# Learning objectives

- Master advanced UVM features and patterns

# Prerequisites

- See module README

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Virtual Sequences and Sequen"]

T2["Coverage Models"]

T1 --> T2

T3["Advanced Configuration"]

T2 --> T3

Master advanced UVM features and patterns

# Overview

- This module covers advanced UVM concepts including virtual sequences, coverage models, advanced con...

# **Design architecture**

# 1. Advanced UVM coordination architecture (1/3)

- Virtual sequences run on virtual sequencer; delegate to agent sequencers
- Register model maps address space — frontdoor (bus) and backdoor (hdl paths)
- Callbacks intercept pre/post drive, report, or phase hooks
- Hierarchy: Test → Env → {Agent(s), Reg model, Coverage collector}

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

A["Add rtl\_architecture.mmd"] --> B["for Module 5"]

# 1. Advanced UVM coordination architecture (2/3)

- `uvm_virtual_sequencer`: Coordinates multiple sub-sequencers (multi-agent)
- `uvm_reg_block` / `uvm_reg`: Register abstraction layer (RAL)
- `uvm_subscriber` / callbacks: Hook component behavior without subclassing
- Coverage model: `covergroup` in classes or standalone cover components

# 1. Advanced UVM coordination architecture (3/3)

- Config objects: Structured uvm\_object passed via ConfigDB

## 2. Cross-cutting concern flow (1/2)

- 1: Config object — Built in test; set via `uvm_config_db`
- 2: Virtual sequence — Starts traffic on multiple interfaces
- 3: Functional coverage — Samples during monitor/scoreboard activity
- 4: Register sequences — RAL write/read with predicted values

Verification Architecture

Diagram: Verification Architecture  
(render with mmdc when available)  
flowchart LR

A["Add verification\_architecture.mmd"] --> B["for Module 5"]

## 2. Cross-cutting concern flow (2/2)

- 5: Callback — Alters or logs transaction before drive

### 3. Module 5 DUT context

- ``dut/advanced/multi_channel.v`` — multi-channel design for virtual sequence demos
- Examples map 1:1 to advanced topics (virtual seq, coverage, reg model, callbacks)
- Integrated test  
``test_advanced_uvm.sv`` combines multiple advanced features

#### Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing\_methods.mmd"] --> B["for Module 5"]

# RTL block diagram (reference)

## Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

A["Add rtl\_architecture.mmd"] --> B["for Module 5"]

Module 5: DUT hierarchy and signal flow.

# Verification / testbench diagram (reference)

## Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

A["Add verification\_architecture.mmd"] --> B["for Module 5"]

Module 5: stimulus, observation, and checking.

# **Execution & simulation flow**

# How the example runs (toolchain)

- Makefile: Verilator + UVM\_HOME compile RTL, interface, and test\_\*.sv
- UVM phases: build → connect → run\_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test\_\* (see module5 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

# Artifact flow (spec → sim) (1)

- 1: Config object — Built in test; set via `uvm_config_db`
- 2: Virtual sequence — Starts traffic on multiple interfaces
- 3: Functional coverage — Samples during monitor/scoreboard activity
- 4: Register sequences — RAL write/read with predicted values

## Artifact flow (spec → sim) (2)

- 5: Callback — Alters or logs transaction before drive

# Example directory layout

- `uvm_virtual_sequencer`: Coordinates multiple sub-sequencers (multi-agent)
- `uvm_reg_block` / `uvm_reg`: Register abstraction layer (RAL)
- `uvm_subscriber` / `callbacks`: Hook component behavior without subclassing
- Coverage model: `covergroup` in classes or standalone cover components
- Config objects: Structured `uvm_object` passed via `ConfigDB`

# Directed test execution sequence

- config
- build
- connect
- virtual seq
- report + coverage

# **Verification & testing methods**

# 1. System-level stimulus strategy

- Virtual sequences coordinate reset, config, traffic, and shutdown across agents
- Constrained-random transactions with covergroups measuring interesting bins
- Register tests: reset values, bit-field access, alias rules, prediction vs actual
- Callbacks for error injection and debug without modifying driver source

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing\_methods.mmd"] --> B["for Module 5"]

## 2. Coverage-driven closure (1/2)

- Define coverpoints on transaction fields and cross coverage between channels
- Use `uvm\_config\_db` to enable/disable coverage and set verbosity per test
- Closure metrics: functional coverage goals + zero scoreboard mismatches
- `./scripts/module5.sh --coverage` demonstrates sampling and report

### Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing\_methods.mmd"] --> B["for Module 5"]

## 2. Coverage-driven closure (2/2)

- Out of scope on Verilator: some advanced RAL backdoor paths may be limited

# Syllabus topics

# Protocol & design details

## Detail: Example 5.1: Virtual Sequences (`module5/examples/virtual\_seq...

- VirtualSequencer: Contains references to multiple sequencers (`master\_seqr`, `slave\_seqr`)
- VirtualSequence: Coordinates sequences on multiple sequencers
- ChannelSequence: Regular sequence for a single channel
- Parallel Execution: Using `fork-join` for concurrent sequences

## Detail: Example 5.1: Virtual Sequences (`module5/examples/virtual\_seq...

- Sequential Execution: Sequential sequence coordination
- Purpose: Hold references to multiple sequencers
- Key: References are set in `connect\_phase()` from environment
- Note: Virtual sequencer doesn't create sequencers, only references them

## Detail: Example 5.2: Coverage (``module5/examples/coverage/coverage.sv...`)

- CoverageModel: Extends ``uvm_subscriber`` for coverage collection
- Coverage Sampling: Via ``write()`` method from analysis port
- Coverage Types: Data, address range, command, and cross coverage
- Coverage Analysis: Coverage statistics and reporting

## Detail: Example 5.2: Coverage

### (`module5/examples/coverage/coverage.sv...

- Purpose: Sample coverage data from transactions
- Usage: Called automatically when monitor writes to analysis port
- Key: Uses associative arrays to track unique values
- Purpose: Track unique values and counts

## Detail: Example 5.3: Advanced Configuration (`module5/examples/config...

- AgentConfig: Complex configuration object with multiple fields
- EnvConfig: Environment-level configuration containing agent configs
- Configuration Hierarchy: Multi-level configuration structure
- Configuration Usage: Setting and getting complex configuration

## Detail: Example 5.3: Advanced Configuration (`module5/examples/config...

- Purpose: Enable configuration copying
- Usage: Allows configuration inheritance and copying
- Key: Must copy all configuration fields
- Purpose: Multi-level configuration hierarchy

## **Detail: Example 5.4: Callbacks** **(`module5/examples/callbacks/callbacks...**

- DriverCallbackBase: Base callback class for driver
- DriverCallback: Driver callback implementation
- MonitorCallbackBase: Base callback class for monitor
- MonitorCallback: Monitor callback implementation

## **Detail: Example 5.4: Callbacks** **(`module5/examples/callbacks/callbacks...**

- Callback Registration: Registering callbacks with components
- Purpose: Define callback interface
- Key: Virtual functions to be overridden in derived classes
- Purpose: Implement callback behavior

## Detail: Example 5.5: Register Model (`module5/examples/register\_model...

- RegisterField: Represents a register field with offset, width, and value
- Register: Represents a register with address and fields
- RegisterBlock: Represents a register block with multiple registers
- Register Operations: Read/write operations on registers

## Detail: Example 5.5: Register Model (`module5/examples/register\_model...

- Purpose: Represent individual register fields
- Key: Fields have offset, width, and value
- Purpose: Add field to register
- Usage: Build register structure

# Detail: Virtual Sequence Methods (1)

- Sequencer References: Hold references to multiple sequencers
- Connect Phase: Set references from environment
- No Transaction Type: Virtual sequencer doesn't extend ``uvm_sequencer #(type)``
- Body Task: Coordinates sequences on multiple sequencers

## Detail: Virtual Sequence Methods (2)

- Sequencer References: Set before starting sequence
- Parallel Execution: ``fork-join`` for concurrent sequences
- Sequential Execution: Sequential ``start()`` calls

# Detail: Coverage Methods (1)

- ``write(t)`` - Receive transaction for sampling
- Associative Arrays - Track unique values
- ``exists(key)`` - Check if key exists
- ``num()`` - Count unique keys

## Detail: Coverage Methods (2)

- ``foreach`` - Iterate over keys

# Detail: Advanced Configuration Methods

- ``do_copy(rhs)`` - Copy configuration fields
- ``convert2string()`` - String representation
- Nested Configs - Config objects containing other configs

# Detail: Callback Methods

- ``uvm_register_cb(Component, Callback)`` - Register callback type
- ``uvm_do_callbacks(Component, Callback, method(args))`` - Execute callbacks
- ``uvm_callbacks#(Component, Callback)::add(component, callback)`` - Add callback
- ``uvm_callbacks#(Component, Callback)::delete(component, callback)`` - Remove callback

# Detail: Register Model Methods (1)

- ``convert2string()`` - String representation
- ``add_field(field)`` - Add field to register
- ``write(data)`` - Write value to register
- ``read()`` - Read value from register

## Detail: Register Model Methods (2)

- ``add_register(reg)`` - Add register to block
- ``get_register(name)`` - Look up register by name
- ``write_register(name, data)`` - Write register by name
- ``read_register(name)`` - Read register by name

# Verification flow (reference)

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing\_methods.mmd"] --> B["for Module 5"]

Stimulus, DUT response, and check points for this module.

# 1. Virtual Sequences and Sequencers (1/9)

- Virtual Sequence Purpose
- Coordinating multiple sequencers
- Cross-agent stimulus generation
- System-level sequence coordination
- Sequence libraries
- Virtual Sequencer Implementation

# 1. Virtual Sequences and Sequencers (2/9)

- Virtual sequencer structure
- Sequencer references
- Connection in connect\_phase
- Virtual sequencer usage
- VirtualSequencer: Contains references to multiple sequencers (``master_seqr``, ``slave_seqr``)
- VirtualSequence: Coordinates sequences on multiple sequencers

# 1. Virtual Sequences and Sequencers (3/9)

- ChannelSequence: Regular sequence for a single channel
- Parallel Execution: Using ``fork-join`` for concurrent sequences
- Sequential Execution: Sequential sequence coordination
- Purpose: Hold references to multiple sequencers
- Key: References are set in ``connect_phase()`` from environment
- Note: Virtual sequencer doesn't create sequencers, only references them

# 1. Virtual Sequences and Sequencers (4/9)

- Purpose: Coordinate sequences on multiple sequencers
- Key: Virtual sequence doesn't extend ``uvm_sequence #(transaction_type)``
- Parallel: Use ``fork-join`` for concurrent execution
- Sequential: Call ``start()`` sequentially for ordered execution
- Purpose: Connect virtual sequencer/sequence to actual sequencers
- Usage: In test's ``connect_phase()``

# 1. Virtual Sequences and Sequencers (5/9)

- Key: Must be set before starting virtual sequence
- Purpose: Start virtual sequence execution
- Key: Virtual sequence can start on virtual sequencer or null
- Purpose: Execute sequences concurrently
- Key: Both sequences run in parallel, `join` waits for both
- Timing: Total time =  $\max(\text{seq1\_time}, \text{seq2\_time})$

# 1. Virtual Sequences and Sequencers (6/9)

- Use case: Independent operations, concurrent testing
- fork-join: Wait for all sequences (most common)
- fork-join\_any: Wait for first to complete
- fork-join\_none: Fire and forget
- Purpose: Execute sequences in order
- Key: Second sequence starts after first completes

# 1. Virtual Sequences and Sequencers (7/9)

- Timing: Total time = seq1\_time + seq2\_time
- Use case: Ordered operations, dependencies
- Purpose: Check sequencer availability
- Key: Always check for null before using sequencer references
- Virtual sequencer contains references to multiple sequencers
- Virtual sequence coordinates sequences on different sequencers

# 1. Virtual Sequences and Sequencers (8/9)

- Parallel sequence execution using fork-join enables concurrent testing
- Sequential sequence execution enables ordered coordination
- Virtual sequencer - Holds references, doesn't create sequencers
- Virtual sequence - Coordinates sequences on multiple sequencers
- Sequencer references - Set in ``connect_phase()` from environment
- Parallel execution - ``fork-join`` for concurrent sequences

# 1. Virtual Sequences and Sequencers (9/9)

- Sequential execution - Sequential ``start()`` calls for ordered execution
- Virtual sequencer creation and connection
- Virtual sequence coordination
- Parallel sequence execution
- Sequential sequence execution

## 2. Coverage Models (1/8)

- Functional Coverage
- Coverage model structure
- Coverage sampling
- Coverage analysis
- Coverage reporting
- Coverage Types

## 2. Coverage Models (2/8)

- Data coverage
- Address range coverage
- Command coverage
- Cross coverage
- CoverageModel: Extends ``uvm_subscriber`` for coverage collection
- Coverage Sampling: Via ``write()`` method from analysis port

## 2. Coverage Models (3/8)

- Coverage Types: Data, address range, command, and cross coverage
- Coverage Analysis: Coverage statistics and reporting
- Purpose: Sample coverage data from transactions
- Usage: Called automatically when monitor writes to analysis port
- Key: Uses associative arrays to track unique values
- Purpose: Track unique values and counts

## 2. Coverage Models (4/8)

- Key: Associative arrays enable efficient coverage tracking
- Purpose: Report coverage statistics
- Usage: Called in cleanup phase
- Key: Uses ``num()`` to count unique keys
- Purpose: Connect producer to coverage model
- Key: Coverage automatically receives via ``write()`` method

## 2. Coverage Models (5/8)

- Purpose: Track combinations of multiple fields
- Key: 2D associative arrays for cross coverage
- Usage: Track (data, command) pairs, (address, data) pairs, etc.
- Purpose: Initialize coverage data structures
- Key: Fixed arrays initialized in constructor
- Coverage model class extending ``uvm_subscriber`` enables automatic sampling

## 2. Coverage Models (6/8)

- Coverage sampling via ``write()`` method receives transactions
- Coverpoints and bins track unique values and ranges
- Cross coverage between multiple fields tracks combinations
- ``write(t)`` - Receive transaction for coverage sampling
- Associative arrays - Track unique values efficiently
- ``exists(key)`` - Check if key exists in associative array

## 2. Coverage Models (7/8)

- ``num()` - Count number of unique keys
- ``foreach`` - Iterate over associative array keys
- 2D associative arrays - Track cross coverage pairs
- Coverage sampling demonstrated
- Data coverage tracking
- Address range coverage

## 2. Coverage Models (8/8)

- Command coverage tracking
- Cross coverage analysis

# 3. Advanced Configuration (1/7)

- Complex Configuration Objects
- Configuration object design
- Configuration fields
- Configuration methods
- Configuration copying
- Configuration Hierarchy

### 3. Advanced Configuration (2/7)

- Environment-level configuration
- Agent-level configuration
- Component-specific configuration
- Configuration inheritance
- AgentConfig: Complex configuration object with multiple fields
- EnvConfig: Environment-level configuration containing agent configs

### 3. Advanced Configuration (3/7)

- Configuration Hierarchy: Multi-level configuration structure
- Configuration Usage: Setting and getting complex configuration
- Purpose: Enable configuration copying
- Usage: Allows configuration inheritance and copying
- Key: Must copy all configuration fields
- Purpose: Multi-level configuration hierarchy

### 3. Advanced Configuration (4/7)

- Key: Configuration objects can contain other configuration objects
- Purpose: Set configuration for specific components
- Key: Use component path in ``set()`` call
- Purpose: Retrieve configuration in component
- Key: Always check return value, provide defaults
- Purpose: Debug configuration values

### 3. Advanced Configuration (5/7)

- Usage: Logging, debugging
- Complex configuration objects enable rich configuration data
- Configuration hierarchy supports multi-level configuration
- Resource database usage provides flexible configuration access
- Configuration inheritance enables configuration reuse
- ``do_copy()` - Enable configuration copying

### 3. Advanced Configuration (6/7)

- Nested configs - Configuration objects containing other configs
- Component-specific paths - Set config for specific components
- Config hierarchy - Multi-level configuration structure
- ``convert2string()`` - Debug configuration values
- Complex configuration object creation
- Configuration hierarchy demonstrated

# 3. Advanced Configuration (7/7)

- Configuration setting and getting
- Configuration object usage

## 4. Callbacks (1/8)

- Callback Mechanism
- Callback base classes
- Callback registration
- Pre/post callbacks
- Callback execution
- Callback Usage Patterns

## 4. Callbacks (2/8)

- Driver callbacks
- Monitor callbacks
- Transaction modification
- Side-effect actions
- DriverCallbackBase: Base callback class for driver
- DriverCallback: Driver callback implementation

## 4. Callbacks (3/8)

- MonitorCallbackBase: Base callback class for monitor
- MonitorCallback: Monitor callback implementation
- Callback Registration: Registering callbacks with components
- Purpose: Define callback interface
- Key: Virtual functions to be overridden in derived classes
- Purpose: Implement callback behavior

## 4. Callbacks (4/8)

- Key: ``uvm_register_cb`` macro registers callback type
- Purpose: Register callback type with component type
- Usage: Must be in callback class definition
- Key: Enables type-safe callback registration
- Purpose: Execute all registered callbacks
- Usage: Call at appropriate points in component code

## 4. Callbacks (5/8)

- Key: Executes all callbacks registered for this component instance
- Purpose: Connect callback to component instance
- Usage: In test's ``connect_phase()`
- Key: Callback must be registered before it can be executed
- Purpose: Manage callbacks
- Key: Static methods operate on callback registry

## 4. Callbacks (6/8)

- Purpose: Multiple callbacks for same component
- Key: All registered callbacks execute in registration order
- Purpose: Control callback execution order
- Key: Pre-callbacks → operation → post-callbacks
- Callback mechanism enables non-intrusive extension
- Pre/post callbacks provide hooks for customization

## 4. Callbacks (7/8)

- Callback registration connects callbacks to components
- Callback usage patterns enable flexible test customization
- ``uvm_register_cb`` - Register callback type
- ``uvm_do_callbacks`` - Execute all registered callbacks
- Callback base class - Define callback interface
- Callback registration - Connect callback to component

## 4. Callbacks (8/8)

- Pre/post callbacks - Hooks before/after operations
- Callback registration demonstrated
- Pre-drive callbacks executed
- Post-drive callbacks executed
- Monitor callbacks executed

# 5. Register Model (1/9)

- Register Model Basics
- Register model structure
- Register blocks, registers, fields
- Register read/write operations
- Register predictor
- Register Model Components

## 5. Register Model (2/9)

- RegisterField: Individual register fields
- Register: Complete registers
- RegisterBlock: Register address spaces
- Register operations
- RegisterField: Represents a register field with offset, width, and value
- Register: Represents a register with address and fields

## 5. Register Model (3/9)

- RegisterBlock: Represents a register block with multiple registers
- Register Operations: Read/write operations on registers
- Purpose: Represent individual register fields
- Key: Fields have offset, width, and value
- Purpose: Add field to register
- Usage: Build register structure

## 5. Register Model (4/9)

- Key: Fields stored in queue
- Purpose: Write value to register
- Key: Updates register value and all field values
- Field extraction: Uses bit shifting and masking
- Purpose: Read register value
- Returns: Register value

## 5. Register Model (5/9)

- Purpose: Add register to block
- Key: Automatically assigns address based on position
- Purpose: Look up register by name
- Key: Returns null if register not found
- Purpose: High-level register access
- Usage: Access registers by name

## 5. Register Model (6/9)

- Key: Handles null checks
- Purpose: Extract field values from register value
- Key: Bit shifting and masking for field extraction
- Example: Field at offset 4, width 3: ``(data >> 4) & 7``
- Purpose: Compose register value from field values
- Key: Shift and OR field values into register

## 5. Register Model (7/9)

- Purpose: Automatically assign register addresses
- Key:  $\text{Address} = \text{base\_address} + (\text{index} * 4)$  for 32-bit registers
- Register model structure provides abstraction for register access
- Register blocks, registers, fields organize register hierarchy
- Register read/write operations enable register verification
- Simplified register model demonstrates concepts (full implementation requires `uvm_reg`)

## 5. Register Model (8/9)

- ``add_field()` - Add field to register
- ``write(data)` - Write value to register
- ``read()` - Read value from register
- ``add_register()` - Add register to block
- ``get_register(name)` - Look up register by name
- ``write_register()` - High-level write by name

## 5. Register Model (9/9)

- ``read_register()` - High-level read by name
- Field extraction - Bit shifting and masking for field values
- Register model structure demonstrated
- Register field operations
- Register read/write operations
- Register block management

## 6. Advanced UVM Methods Reference (1/5)

- Sequencer References: Hold references to multiple sequencers
- Connect Phase: Set references from environment
- No Transaction Type: Virtual sequencer doesn't extend ``uvm_sequencer #(type)``
- Body Task: Coordinates sequences on multiple sequencers
- Sequencer References: Set before starting sequence
- Parallel Execution: ``fork-join`` for concurrent sequences

## 6. Advanced UVM Methods Reference (2/5)

- Sequential Execution: Sequential ``start()` calls
- ``write(t)` - Receive transaction for sampling
- Associative Arrays - Track unique values
- ``exists(key)` - Check if key exists
- ``num()` - Count unique keys
- ``foreach` - Iterate over keys

## 6. Advanced UVM Methods Reference (3/5)

- ``do_copy(rhs)`` - Copy configuration fields
- ``convert2string()`` - String representation
- Nested Configs - Config objects containing other configs
- ``uvm_register_cb(Component, Callback)`` - Register callback type
- ``uvm_do_callbacks(Component, Callback, method(args))`` - Execute callbacks
- ``uvm_callbacks#(Component, Callback)::add(component, callback)`` - Add callback

## 6. Advanced UVM Methods Reference (4/5)

- ``uvm_callbacks#(Component, Callback)::delete(component, callback)`` - Remove callback
- ``convert2string()`` - String representation
- ``add_field(field)`` - Add field to register
- ``write(data)`` - Write value to register
- ``read()`` - Read value from register
- ``add_register(reg)`` - Add register to block

## 6. Advanced UVM Methods Reference (5/5)

- ``get_register(name)`` - Look up register by name
- ``write_register(name, data)`` - Write register by name
- ``read_register(name)`` - Read register by name

## Run: Example 5.1: Virtual Sequences

```
./scripts/module5.sh --virtual-sequences
cd module5/examples/virtual_sequences
verilator -sv --cc --exe --timing --trace \
-I$UVM_HOME/src +incdir+$UVM_HOME/src +define+UVM_NO_DPI \
--binary $UVM_HOME/src/uvm_pkg.sv virtual_sequences.sv
virtual_sequences.cpp \
--top-module virtual_sequences
make -C obj_dir -f Vvirtual_sequences.mk
./obj_dir/Vvirtual_sequences
```

## Run: Example 5.2: Coverage

```
./scripts/module5.sh --coverage
```

# Run: Example 5.3: Advanced Configuration

```
./scripts/module5.sh --configuration
```

## Run: Example 5.4: Callbacks

```
./scripts/module5.sh --callbacks
```

# Run: Example 5.5: Register Model

```
./scripts/module5.sh --register-model
```

# Hands-on examples

# Module 5 self-check

```
$ ./scripts/module5.sh --help
```

Terminal: module5\_check

Usage: ./scripts/module5.sh [OPTIONS]

Module 5: Advanced UVM Concepts

This script runs examples and tests for Module 5.

## OPTIONS:

### UVM Examples:

- virtual-sequences Run virtual sequence examples
- coverage Run coverage examples
- configuration Run configuration examples
- callbacks Run callback examples
- register-model Run register model examples
- all-examples Run all UVM examples (default)
- skip-examples Skip all UVM examples

### Tests:

- uvm-tests Run UVM testbenches
- all-tests Run all tests

### Environment:

- sim SIMULATOR Simulator to use (default: verilator)
- uvm-home DIR UVM library directory (default: \$UVM\_HOME)
- jobs N Number of parallel build jobs (default: 8)
- no-clean Skip cleaning before build (faster rebuilds)

### Other:

- help, -h Show this help message

## EXAMPLES:

- # Run all UVM examples (with parallel builds)
- ./scripts/module5.sh

- # Run only virtual sequences
- ./scripts/module5.sh --virtual-sequences

- # Run UVM tests

# Exercise scaffold

```
./scripts/module5.sh --scaffold
```

# Example 1: Virtual Sequences

- VirtualSequencer: Contains references to multiple sequencers (``master_seqr``, ``slave_seqr``)
- VirtualSequence: Coordinates sequences on multiple sequencers
- ChannelSequence: Regular sequence for a single channel
- Parallel Execution: Using ``fork-join`` for concurrent sequences
- Sequential Execution: Sequential sequence coordination

# Demo: Virtual Sequences

```
$ ./scripts/module5.sh --virtual-sequences
```

Terminal: ex01\_virtual\_sequences

```
[0:36m===== [0m
[0:36mModule 5: Advanced UVM Concepts [0m
[0:36m===== [0m

[0:34m[2026-05-31 03:57:01] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 03:57:01] Checking prerequisites... [0m
[0:31m[2026-05-31 03:57:01] Error: verilator not found. Please install it first. [0m
```

## Example 2: Coverage

- CoverageModel: Extends ``uvm_subscriber`` for coverage collection
- Coverage Sampling: Via ``write()`` method from analysis port
- Coverage Types: Data, address range, command, and cross coverage
- Coverage Analysis: Coverage statistics and reporting

# Demo: Coverage

```
$ ./scripts/module5.sh --coverage
```

Terminal: ex02\_coverage

```
[0:36m===== [0m
[0:36mModule 5: Advanced UVM Concepts [0m
[0:36m===== [0m

[0:34m[2026-05-31 03:57:01] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 03:57:01] Checking prerequisites... [0m
[0:31m[2026-05-31 03:57:01] Error: verilator not found. Please install it first. [0m
```

## Example 3: Advanced Configuration

- AgentConfig: Complex configuration object with multiple fields
- EnvConfig: Environment-level configuration containing agent configs
- Configuration Hierarchy: Multi-level configuration structure
- Configuration Usage: Setting and getting complex configuration

# Demo: Advanced Configuration

```
$ ./scripts/module5.sh --configuration
```

Terminal: ex03\_configuration

```
[0:36m===== [0m
[0:36mModule 5: Advanced UVM Concepts [0m
[0:36m===== [0m

[0:34m[2026-05-31 03:57:02] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 03:57:02] Checking prerequisites... [0m
[0:31m[2026-05-31 03:57:02] Error: verilator not found. Please install it first. [0m
```

## Example 4: Callbacks

- DriverCallbackBase: Base callback class for driver
- DriverCallback: Driver callback implementation
- MonitorCallbackBase: Base callback class for monitor
- MonitorCallback: Monitor callback implementation
- Callback Registration: Registering callbacks with components

# Demo: Callbacks

```
$ ./scripts/module5.sh --callbacks
```

Terminal: ex04\_callbacks

```
[0:36m===== [0m
[0:36mModule 5: Advanced UVM Concepts [0m
[0:36m===== [0m

[0:34m[2026-05-31 03:57:02] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 03:57:02] Checking prerequisites... [0m
[0:31m[2026-05-31 03:57:02] Error: verilator not found. Please install it first. [0m
```

# Example 5: Register Model

- RegisterField: Represents a register field with offset, width, and value
- Register: Represents a register with address and fields
- RegisterBlock: Represents a register block with multiple registers
- Register Operations: Read/write operations on registers

# Demo: Register Model

```
$ ./scripts/module5.sh --register-model
```

Terminal: ex05\_register\_model

```
[0:36m===== [0m
[0:36mModule 5: Advanced UVM Concepts [0m
[0:36m===== [0m

[0:34m[2026-05-31 03:57:02] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 03:57:02] Checking prerequisites... [0m
[0:31m[2026-05-31 03:57:03] Error: verilator not found. Please install it first. [0m
```

# Practice & assessment

# What you should know

- Implement virtual sequences to coordinate multiple sequencers
- Create coverage models for functional verification
- Use advanced configuration with complex objects
- Implement callbacks for component extension
- Use register models for register verification
- Apply advanced UVM patterns in testbenches

# Exercises

- Virtual Sequences
- Coverage Models
- Advanced Configuration
- Callbacks
- Register Model

# Assessment checklist

- Can implement virtual sequences
- Can create coverage models
- Can use advanced configuration
- Can implement callbacks
- Can use register models
- Understands advanced UVM patterns

# Summary & next steps

- Master advanced UVM features and patterns
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: Next module in course